We're not going to go over them all here, but there are a few we'd like to highlight.

## Environment Options

Sometimes, you need to control how RSpec loads your Ruby code. For instance, you may be experimenting with a locally modified version of a library—in that case, you'd want RSpec to load your custom version of that library instead of the default one.

The first two options listed in the --help output are for these kinds of environment customizations:

```
-I PATH                          Specify PATH to add to $LOAD_PATH (may ↩
                                 be used more than once).
-r, --require PATH               Require a file.
```

If you've ever passed the -I or -r options to the ruby executable, these RSpec switches may look familiar—they were designed to match the ones for Ruby.

The second option, -r or --require, makes it easy to use supporting libraries while you're testing. For example, you may want to use the byebug debugger to troubleshoot a failing spec.[1] You can easily enable debugging for a single RSpec run by using the -r option together with the library name:

```
$ rspec -rbyebug
```

The other option in this group, -I, adds a directory to Ruby's load path.[2] This helps Ruby find libraries that you load from a require statement or the --require switch.

RSpec already adds the two most important directories to the load path: your project's lib and spec folders. But sometimes you may want to use a particular library without going through Bundler or RubyGems; that's where this flag comes in handy.

## Filtering Options

Now that we've covered options that configure the environment for your specs, let's talk about the options that govern which of your specs RSpec will run. Running just the specs you need at any given time will make you much more productive. To that end, RSpec supports a number of filtering options, listed further down in the --help output:

1. https://github.com/deivid-rodriguez/byebug
2. http://webapps-for-beginners.rubymonstas.org/libraries/load_path.html